LEUKEMIA HARMONISER — ULTRA-CONDENSED

🎯 WHAT IT DOES

Ingests CBC (blood count) datasets with messy column names → harmonises them → runs 3 ML models → outputs: Leukemia Positive / Inconclusive / Healthy per patient.

🔄 DATA FLOW (Top to Bottom)

```
1. USER UPLOADS CSV
   ↓
2. HARMONISATION ENGINE
   • Fuzzy match columns to 17 canonical CBC features (WBC, RBC, Hemoglobin, etc.)
   • Fix unit scales (10³/L → cells/μL if needed)
   • Clean outliers using clinical ranges
   • Fill nulls with population defaults
   • Result: cleaned DataFrame, zero rows dropped
   ↓
3. ML PREDICTION
   • 3 ensemble models: XGBoost + Random Forest + Neural Network
   • Each outputs probability P(Leukemia | CBC)
   • Convert to vote: if P ≥ 0.20 → vote=1 (positive), else vote=0 (negative)
   ↓
4. CONSENSUS VOTING
   • All 3 vote 1? → HIGH RISK (leukemia positive)
   • All 3 vote 0? → LOW RISK (healthy)
   • Mixed votes? → INCONCLUSIVE (needs further testing)
   ↓
5. RESULTS STORED & DISPLAYED
   • Patient table with predictions from each model
   • Risk flags, probabilities, model disagreements
   • PDF reports on demand
```

🔬 WHY EACH STAGE EXISTS

Stage	Why
Fuzzy Column Matching	Hospital labs use different column names (WBC_Count_ul vs white_blood_cell_count) — need to map to standard names

Stage	Why
Scale Correction	5000 cells/μL = 5 × 10^9/L (same value, different units) — must normalize to one unit before ML
Outlier Cleaning	Values outside clinical ranges = measurement error (e.g., RBC=50) — replace with median
Null Filling	Models can't handle NaN → fill with normal population median
3 Models Ensemble	Single model can be overconfident → 3 models give consensus vote (reduces false alarms)
Consensus Voting	If 2/3 models agree = high confidence; if 1/3 disagree = mark inconclusive (clinician decides)

📦 LIBRARIES (WHAT EACH DOES)

Data & Math

Library	Use
`pandas`	Read CSV, manipulate DataFrames
`numpy`	Array math, statistics (median)
`rapidfuzz`	Fuzzy string matching for column names (72% threshold)

Machine Learning

Library	Model	Why Used
`xgboost`	XGBoost	Sequential error correction, fast, feature importance, best on tabular data
`sklearn.ensemble`	Random Forest	Non-linear, parallel trees, robust baseline
`sklearn.neural_network`	MLP (3-layer NN)	Deep learning, pattern recognition
`imblearn.SMOTE`	Synthetic oversampling	Generates fake positive cases to balance imbalanced data (rare disease)

Library	Model	Why Used
`sklearn.preprocessing`	StandardScaler, LabelEncoder	Normalize features & encode gender
`sklearn.metrics`	accuracy, ROC-AUC, F1, precision, recall	Evaluate model performance

Web & API

Library	Use
`fastapi`	REST API framework (endpoints: /api/upload, /api/predict, etc.)
`uvicorn`	HTTP server to run FastAPI
`pydantic`	Validate request/response data
`pyngrok`	Create secure HTTPS tunnel to expose localhost to internet

Files & Reporting

Library	Use
`joblib`	Save/load trained models as .pkl files
`reportlab`	Generate PDF reports (tables, styles, colors)
`io.BytesIO`	In-memory file buffers (for CSV parsing & PDF output)

Frontend (No Dependencies)

Tech	Use
HTML/CSS/JS (vanilla)	Dashboard UI, no frameworks needed
Canvas API	Draw pie/bar charts (no charting library)
Fetch API	Call REST endpoints from browser

System

Library	Use
`threading`	Run training jobs in background (non-blocking UI)

Library	Use
`asyncio`	Async/await for background tasks
`nest_asyncio`	Patch event loop for Colab environment

🎯 CLASSIFICATION LOGIC (THE CORE)

How a Patient Gets Classified

```
CBC Input: [WBC=45000, RBC=2.5, Hemoglobin=6.0, Platelets=15000, ...]
                                ↓
          Feature Extraction & Normalization
                                ↓
        Run through 3 trained models:

        Model 1 (XGBoost):      P(Leukemia) = 0.85 → vote = 1
        Model 2 (Random Forest): P(Leukemia) = 0.72 → vote = 1
        Model 3 (Neural Net):    P(Leukemia) = 0.68 → vote = 1
                                ↓
                  Consensus: 3/3 = ALL 1
                                ↓
                  Risk = HIGH RISK ✓
```

Decision Rules

python

```
Threshold = 0.20  # If P(Leukemia) >= 0.20 → vote=1, else vote=0

Risk Tier:
├─ All 3 vote 1    → "Positive" / "High Risk" (leukemia)
├─ All 3 vote 0    → "Negative" / "Low Risk" (healthy)
└─ Mixed (1-2 vote 1) → "Inconclusive" (further testing needed)

Probability Ranges (for dashboard info):
├─ P >= 0.70       → High confidence positive
├─ 0.30 < P < 0.70 → Borderline/inconclusive
└─ P < 0.30        → High confidence negative
```

🏥 WHAT TRIGGERS EACH CLASSIFICATION

Leukemia Positive (HIGH RISK)

CBC Pattern: **Elevated WBC** (>20k) + **Low RBC** (<3.5) + **Low Hemoglobin** (<7) + **Low Platelets** (<20k)

- Models detect this as abnormal marrow invasion
- All 3 models vote positive → consensus "POSITIVE"
- Risk = HIGH

Inconclusive (WATCH LIST)

CBC Pattern: **Some abnormal values** but not all, **or models disagree**

- E.g., high WBC but normal hemoglobin
- Or: 2 models say positive, 1 says negative
- Consensus = "INCONCLUSIVE"
- Risk = INCONCLUSIVE (requires follow-up)

Healthy (LOW RISK)

CBC Pattern: **All values in normal range**

- WBC 4.5-11k, RBC 4.5-5.5, Hemoglobin 13.5-17.5, Platelets 150-400k, etc.
- Models detect as normal
- All 3 models vote negative → consensus "NEGATIVE"
- Risk = LOW

🖥️ UI FLOW (7 Pages)

Page	What It Does
Overview	KPI cards (total records, positive count, models loaded) + charts
Data Intake	Upload CSV → auto-harmonise → show confidence % & dropped columns
Harmonisation	Show mapping table (which original column → which CBC field), confidence score
Model Training	Load pre-trained models OR train new ones on labelled datasets
Analysis Dashboard	Risk distribution chart + patient table (filterable, paginated)

Page	What It Does
Patient Records	Patient cards (3-column grid) + model disagreement highlights
Reports	Generate PDF report (executive summary + statistics + high-risk patients)

⚙️ API ENDPOINTS (Backend)

Method	Endpoint	Does
POST	`/api/upload`	Accept CSV file → harmonize → store
POST	`/api/predict/{ds}`	Run 3 models on harmonized dataset
GET	`/api/predictions/{ds}`	Get paginated patient predictions
POST	`/api/train`	Start background training job
GET	`/api/train/status/{jobid}`	Poll training progress
POST	`/api/report`	Generate & download PDF
GET	`/api/overview`	Get KPI stats

🔑 KEY THRESHOLDS

```
Fuzzy Match Threshold      = 72%      (catch typos without false positives)
SMOTE k_neighbors          = 5        (generate synthetic positives)
Probability Threshold      = 0.20     (low = catch more positives)
High Risk Boundary         = 0.70     (confident positive)
Inconclusive Range         = 0.30-0.70 (borderline)
Test/Train Split           = 80/20    (with stratification)
XGBoost Learning Rate      = 0.1      (conservative updates)
XGBoost Tree Depth         = 6        (shallow, prevents overfitting)
Neural Net Early Stopping  = 15 epochs (if no improvement)
```

🚀 RUNTIME ARCHITECTURE

```
Google Colab (Python runtime)
    ├─ Cell 1: Install deps + load pre-trained models
```

```
    └─ Cell 2: Start system
        ├─ FastAPI app initialized
        ├─ Uvicorn server started on port 8000 (in dedicated thread)
        ├─ ngrok tunnel created (exposes to public HTTPS URL)
        └─ Frontend HTML embedded in FastAPI

    User navigates to: https://xxxxx.ngrok.io

    Browser ↔ FastAPI REST API ↔ In-memory state dicts
                                  └─ MODELS, RAW_DATASETS, PROCESSED,
PREDICTIONS
```

📊 CRITICAL DESIGN DECISIONS

Decision	Why
Ensemble (3 models)	Single model unreliable on CBC data; 3 gives consensus
Zero row loss	Harmonisation never drops data; fills missing with medians
SMOTE for balancing	Leukemia is rare; SMOTE generates synthetic positives
Threshold 0.20	Low threshold = catch more positives (recall > precision)
Consensus voting	"All 3 agree" = high confidence; "mixed" = flag as inconclusive
Canvas charts	No CDN (CDN fails over ngrok); all charts drawn in JS
Background training	Training runs in thread; UI stays responsive
In-memory storage	All data kept in RAM; instant API response (no DB latency)

📈 PERFORMANCE

Operation	Time
Upload & harmonize 8k rows	~2s
Run 3-model predictions on 8k rows	~0.5s
Train XGBoost (200 trees)	~10-15s
Generate PDF report	~3-5s

Operation	Time
Dashboard load	<100ms

That's it. Full project in ~500 words. Everything top-to-bottom.